

```

1 // Define (p0, v0, t0), ..., (p4, v4, t4) as in Figure 5.1.
2 StateData state0(p0, v0, t0); // Defined similarly for all other states.
3 // Assume all the poses are in SE(3).
4
5 // Suppose we have several unary factors (e.g., priors or noisy
6   measurements) on p0, p3.
7 const Pose3 p0PriorValue( Rot3(), Point3(0.0, 0.0, 0.0) );
8 const Pose3 p3PriorValue1(Rot3(), Point3(1.0, 0.2, 0.0));
9 const Pose3 p3PriorValue2(Rot3(), Point3(1.0, -0.1, 0.0));
10
11 const auto unaryNoiseModel =
12   noiseModel::Diagonal::Sigmas(Vector6::Ones());
13 const auto unaryFactor1 = std::make_shared<PriorFactor<Pose3>>(
14   p0, p0PriorValue, unaryNoiseModel);
15 const auto unaryFactor2 = std::make_shared<PriorFactor<Pose3>>(
16   p3, p3PriorValue1, unaryNoiseModel);
17 const auto unaryFactor3 = std::make_shared<PriorFactor<Pose3>>(
18   p3, p3PriorValue2, unaryNoiseModel);
19
20 // In addition, suppose all poses are connected via WNOA factors.
21
22 // Factor graph visualization before interpolation:
23 // unary          unary
24 // |              |
25 // 0 ---- 1 ---- 2 ---- 3 ---- 4
26 //  b      i      b      i      b
27 //              |
28 //              unary
29
30 // Define bordering and interpolated states
31 set<StateData> borderStates = {state0, state2, state4};
32 set<StateData> interpStates = {state1, state3};
33
34 // Define the interpolation factors to wrap the two original factors.
35 auto Q_se3 = Vector6::Ones(); // Power spectral density
36 const auto unaryFactorWrapped1 = WnoaInterpFactor<Pose3>(
37   unaryFactor2, borderStates, interpStates, Q_se3);
38 const auto unaryFactorWrapped2 = WnoaInterpFactor<Pose3>(
39   unaryFactor3, borderStates, interpStates, Q_se3);
40
41 // Define interpolated to border states mapping
42 unordered_map<StateData, pair<StateData, StateData>> interp_to_borders;
43 interp_to_borders[state1] = pair(state0, state2);
44 interp_to_borders[state3] = pair(state2, state4);
45
46 // Create graph.
47 WnoaFactorGraph<Pose3> graphInter(interp_to_borders, Q_se3);
48
49 graphInter.add(unaryFactor1);
50 graphInter.add(unaryFactorWrapped1);
51 graphInter.add(unaryFactorWrapped2);
52
53 // Pass to optimizer.
54 Values initial = ...
55 LevenbergMarquardtParams opt_params;
56
57 Values result = LevenbergMarquardtOptimizer<Pose3>(graphInter, initial,
58   opt_params).optimize();

```